

## Import & Setup

```
import pandas as pd
import numpy as np
```

```
# Display options
pd.set_option('display.max_rows', 100)
pd.set_option('display.max_columns', 10)
pd.set_option('display.width', None)
```

## DataFrame Creation

```
# From dictionary
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': ['x', 'y', 'z']
})

# From lists
df = pd.DataFrame(data, columns=['A', 'B'])

# From numpy array
df = pd.DataFrame(np.random.randn(4, 3),
                  columns=['A', 'B', 'C'])
```

```
# Empty DataFrame
df = pd.DataFrame()
```

## Data Loading/Saving

```
# CSV
df = pd.read_csv('file.csv')
df.to_csv('output.csv', index=False)

# Excel
df = pd.read_excel('file.xlsx')
df.to_excel('output.xlsx', index=False)

# JSON
df = pd.read_json('file.json')
df.to_json('output.json')

# SQL
df = pd.read_sql('SELECT * FROM table', con)

# Parquet
df = pd.read_parquet('file.parquet')
df.to_parquet('output.parquet')
```

## Data Inspection

```
df.head(n=5)          # First n rows
df.tail(n=5)          # Last n rows
df.sample(n=5)        # Random n rows
df.shape              # (rows, columns)
df.info()             # Data types & memory
df.describe()         # Summary statistics
df.dtypes             # Column data types
df.columns            # Column names
df.index              # Row indices
df.nunique()          # Unique values per column
```

```
df.isnull().sum()     # Missing values count
df.memory_usage()     # Memory usage
```

## Data Selection

```
# Single column
df['A']
df.A

# Multiple columns
df[['A', 'B']]

# Rows by position
df.iloc[0]            # First row
df.iloc[0:3]          # First 3 rows
df.iloc[:, 0:2]       # First 2 columns

# Rows by label
df.loc['index_name']
df.loc[df['A'] > 5]    # Conditional

# Boolean indexing
df[df['A'] > 5]
df[(df['A'] > 5) & (df['B'] < 10)]
df[df['A'].isin([1, 2, 3])]

# Query method
df.query('A > 5 and B < 10')
```

## Data Cleaning

```
# Drop missing values
df.dropna()           # Drop rows
df.dropna(axis=1)     # Drop columns
df.dropna(subset=['A', 'B']) # Specific cols

# Fill missing values
df.fillna(0)          # Fill with 0
df.fillna(method='ffill') # Forward fill
df.fillna(method='bfill') # Backward fill
df.fillna(df.mean())  # Fill with mean

# Drop duplicates
df.drop_duplicates()
df.drop_duplicates(subset=['A'])

# Replace values
df.replace(0, np.nan)
df.replace({'A': {1: 100}})

# Rename columns
df.rename(columns={'old': 'new'})
df.columns = ['new1', 'new2']

# Change data types
df['A'] = df['A'].astype('int64')
df = df.astype({'A': 'int64', 'B': 'str'})
```

## Data Manipulation

```
# Add columns
df['C'] = df['A'] + df['B']
df = df.assign(C=df['A'] + df['B'])

# Apply functions
df['A'].apply(lambda x: x**2)
df.apply(lambda row: row['A'] + row['B'],
         axis=1)

# Map values
df['A'].map({1: 'one', 2: 'two'})

# Sort
df.sort_values('A')
df.sort_values(['A', 'B'], ascending=False)
df.sort_index()

# Reset index
df.reset_index(drop=True)

# Set index
df.set_index('A')

# Transpose
df.T

# Melt (wide to long)
pd.melt(df, id_vars=['A'],
        value_vars=['B', 'C'])

# Pivot (long to wide)
df.pivot(index='A', columns='B',
         values='C')
```

## String Operations

```
# String methods (use .str accessor)
df['A'].str.upper()
df['A'].str.lower()
df['A'].str.contains('pattern')
df['A'].str.startswith('prefix')
df['A'].str.endswith('suffix')
df['A'].str.split('delimiter')
df['A'].str.replace('old', 'new')
df['A'].str.strip() # Remove whitespace
df['A'].str.len() # String length
df['A'].str.slice(0, 3) # Substring
```

## Grouping & Aggregation

```
# Basic groupby
df.groupby('A').sum()
df.groupby('A').mean()
df.groupby('A').count()
df.groupby('A').agg(['sum', 'mean', 'count'])

# Multiple columns
df.groupby(['A', 'B']).sum()

# Custom aggregation
df.groupby('A').agg({
    'B': 'sum',
    'C': ['mean', 'std']
})

# Transform (keep original shape)
df.groupby('A').transform('mean')

# Filter groups
df.groupby('A').filter(lambda x: len(x) > 2)

# Apply custom function
df.groupby('A').apply(lambda x: x.max() - x.min())
```

## Merging & Joining

```
# Merge DataFrames
pd.merge(df1, df2, on='key')
pd.merge(df1, df2, left_on='A', right_on='B')
pd.merge(df1, df2, how='left') # left join
pd.merge(df1, df2, how='right') # right join
pd.merge(df1, df2, how='outer') # full outer
pd.merge(df1, df2, how='inner') # inner join

# Join on index
df1.join(df2)
df1.join(df2, how='outer')

# Concatenate
pd.concat([df1, df2]) # Vertically
pd.concat([df1, df2], axis=1) # Horizontally
pd.concat([df1, df2], ignore_index=True)
```

```
# Append (deprecated, use concat)
# df1.append(df2)
```

## Statistical Operations

```
# Descriptive statistics
df.mean() # Column means
df.std() # Standard deviation
df.var() # Variance
df.min() # Minimum values
df.max() # Maximum values
df.median() # Median values
df.quantile(0.25) # 25th percentile
df.corr() # Correlation matrix
df.cov() # Covariance matrix

# Count operations
df.count() # Non-null counts
df.value_counts() # Value frequency (Series)
df['A'].value_counts()
```

```
# Cumulative operations
df.cumsum() # Cumulative sum
df.cummax() # Cumulative maximum
df.cummin() # Cumulative minimum
df.cumprod() # Cumulative product
```

```
# Rolling window operations
df.rolling(window=3).mean()
df.rolling(window=5).std()
df.expanding().mean() # Expanding window
```

## Date/Time Operations

```
# Convert to datetime
df['date'] = pd.to_datetime(df['date'])
df['date'] = pd.to_datetime(df['date'],
                             format='%Y-%m-%d')

# Date components
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day
df['weekday'] = df['date'].dt.day_name()

# Date range
dates = pd.date_range(start='2023-01-01',
                       end='2023-12-31', freq='D')

# Set datetime index
df.set_index('date', inplace=True)

# Resample time series
df.resample('M').mean() # Monthly mean
df.resample('Q').sum() # Quarterly sum

# Time zone operations
df['date'].dt.tz_localize('UTC')
df['date'].dt.tz_convert('US/Eastern')
```

## Categorical Data

```
# Create categorical
df['cat'] = df['A'].astype('category')
df['cat'] = pd.Categorical(df['A'])

# Category operations
df['cat'].cat.categories
df['cat'].cat.codes
df['cat'].cat.add_categories(['new_cat'])
df['cat'].cat.remove_categories(['old_cat'])
df['cat'].cat.rename_categories({'old': 'new'})

# Order categorical
df['cat'] = df['cat'].cat.reorder_categories(
    ['low', 'medium', 'high'], ordered=True)
```

## Pivot Tables & Crosstabs

```
# Pivot table
pd.pivot_table(df, values='C', index='A',
               columns='B', aggfunc='mean')

# Multiple aggregation functions
pd.pivot_table(df, values='C', index='A',
               columns='B',
               aggfunc=['sum', 'mean'])

# Crosstab
pd.crosstab(df['A'], df['B'])
pd.crosstab(df['A'], df['B'], normalize=True)
pd.crosstab(df['A'], df['B'], margins=True)
```

## Advanced Indexing

```
# MultiIndex
df.set_index(['A', 'B'])
df.reset_index()

# Level operations
df.swaplevel()
df.reorder_levels([1, 0])

# Index slicing
df.loc[('A1', 'B1')] # Single tuple
df.loc[('A1', 'B1'):(('A2', 'B2'))] # Range

# Cross-section
df.xs('A1', level=0)
df.xs('B1', level=1)

# Index operations
df.index.get_level_values(0)
df.index.names = ['level1', 'level2']

# Performance Tips
# Use vectorized operations
df['C'] = df['A'] + df['B'] # Good
# df.apply(lambda x: x['A'] + x['B'], axis=1) #
# Slower
```

```
# Use .loc for label-based indexing
df.loc[df['A'] > 5, 'B'] = 0

# Use .iloc for position-based indexing
df.iloc[0:5, 1:3]

# Chain operations efficiently
result = (df
          .query('A > 5')
          .groupby('B')
          .sum()
          .reset_index())

# Use categorical for repeated strings
df['category'] = df['category'].astype('category')

# Read CSV efficiently
df = pd.read_csv('file.csv',
                 dtype={'A': 'int32'},
                 usecols=['A', 'B', 'C'])
```

## Visualization (Basic)

```
# Matplotlib backend (default)
import matplotlib.pyplot as plt

# Basic plots
df.plot() # Line plot
df.plot(kind='bar') # Bar plot
df.plot(kind='hist') # Histogram
df.plot(kind='box') # Box plot
df.plot(kind='scatter', x='A', y='B')

# Subplots
df.plot(subplots=True)

# Save plot
df.plot()
plt.savefig('plot.png')
plt.show()
```

```
# Quick stats visualization
df.hist() # Histograms for all
          columns
pd.plotting.scatter_matrix(df) # Scatter matrix

# Common Patterns

# Check for missing values
df.isnull().any()
df.isnull().sum()

# Filter and select
filtered = df[df['A'] > 5][['B', 'C']]

# Group and aggregate
summary = (df.groupby('category')
           .agg({'price': 'mean',
                 'quantity': 'sum'})
           .round(2))

# Rank values
df['rank'] = df['score'].rank(ascending=False)

# Binning continuous variables
df['age_group'] = pd.cut(df['age'],
                        bins=[0, 18, 35, 65,
                              100],
                        labels=['Child', 'Young',
                                'Adult', 'Senior'])

# One-hot encoding
pd.get_dummies(df['category'])

# Conditional assignment
df['label'] = np.where(df['score'] > 80,
                      'Pass', 'Fail')

# Top N values
df.nlargest(10, 'score')
df.nsmallest(5, 'price')
```